

GPTAid: Automated Generation of API Parameter Security Rules via Execution Feedback and LLM Reasoning – Upgraded

Stanley Chung
Columbia University, New York, NY
Email: syc2153@columbia.edu

Abstract—Misuse of library APIs often results in security vulnerabilities, such as null pointer dereferencing and memory corruption. Although API Parameter Security Rules (APSRs) can mitigate these risks, manually constructing them is labor-intensive. Existing automated approaches often suffer from incomplete documentation or rigid heuristics. This project proposes an upgrade of GPTAid, a LLM-based framework that dynamically retrieves external functions, analyzes API source code, and generates concrete APSRs. The upgrade includes 2 components, dynamic code retrieval and multi-agent rule refinement, designed to resolve the main limitations of the original framework: lack of context and insufficient rule refinement process.

I. PROBLEM STATEMENT AND MOTIVATION

Application Programming Interfaces (APIs) are the building blocks of modern software reuse, and their safe invocation requires strict adherence to security constraints, namely Parameter Security Rules (APSRs). Violations of these rules account for about 71% of known API misuses [1]. Despite their importance, APSRs are frequently undocumented due to its labor-intensive nature to generate. The original GPTAid paper [1] mentions that over 60% of necessary security rules lack explicit descriptions in official documentation.

GPTAid was developed to solve the aforementioned problem. By passing the API sourcecode into the 3-stage pipeline, GPTAid first generates unverified APSRs based on the code, creates correct and violation code to verify them, and refine and finalize them, as shown in Figure 1. As a result, GPTAid

achieved a 92.3% precision, and helped identify 150 issues confirmed the developers. Despite great performance, the framework appears outdated in 2026 due to 2 limitations: The lack of context and incorrect key identification.

The Lack of Context: A lot of APIs are "function wrappers", meaning the API invokes external function within its body. However, the original GPTAid framework only feeds the LLM the top layer of the API and does not provide any information about the external functions besides their function signatures. This significantly limits the quality of rules generated by LLMs in the first stage because the function name itself provides very little information that helps the analysis, and the LLM can only rely on stereotypical inferences.

Incorrect Key Identification: According to the paper, 75% of false positives resulted from incorrect key operation identification during APSRs refinement, where LLM needs to identify the key operations by comparing correct and violation codes and generate the APSRs related to the key operations. However, in cases with intricate modifications, the LLM often misidentifies key operations, leading to false positives. [1]

To address these challenges, we propose 2 enhancements to the framework: The **Bounded Inter-procedural Retrieval Layer** in stage 1, and the **Critic Agent** in stage 3, described in the next section.

II. PROPOSED APPROACH: THE ENHANCEMENTS

This section dives into the details of the **Bounded Inter-procedural Retrieval Layer** and the **Critic Agent** components.

A. Bounded Inter-procedural Retrieval Layer

As shown in Fig. 2, the layer dynamically retrieves the code of external functions within APIs from the pre-downloaded code base. This procedure is bounded by a parameter $k \leq 2$ to avoid context explosion. The code base is pre-downloaded rather than dynamically retrieved online to isolate the layer and ensure our finding directly reflects the efficacy of the component.

B. Critic Agent

We transform the 3rd stage of the original system into a Multi-Agent one by adding the Critic Agent, which acts as a "verifier" to check the quality of proposed APSR (See Fig.

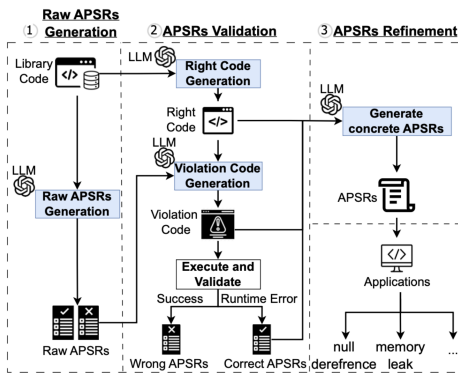


Fig. 1. The Original GPTAid

generated 6 times more APSRs than state-of-the-art detectors,

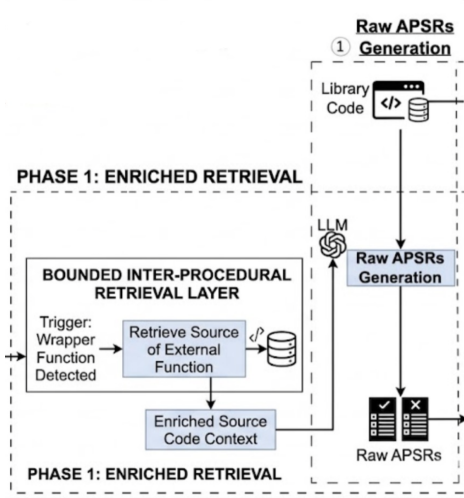


Fig. 2. The Bounded Inter-Procedural Retrieval Layer

3). The Critic agent takes the proposed rule and the correct and violation code as input, performs code difference analysis, and checks if the APSR matches the code difference. The APSR will be approved if it matches the code difference and discarded otherwise. A feedback loop is not implemented for invalid proposals due to budget limitations.

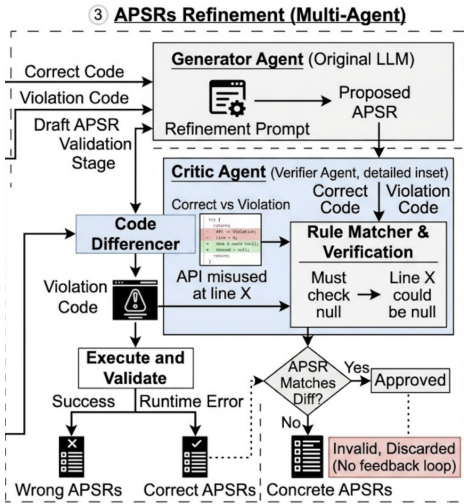


Fig. 3. The New Stage 3

III. EVALUATION PLAN

A. Dataset Construction

We will evaluate the framework with the set of APIs and ground truth data used in the original paper to provide a direct performance comparison.

B. Metrics

- **Precision & Recall:** Measuring the accuracy of generated APSRs against manually verified rules
- **Bug Detection Rate:** The number of confirmed unknown vulnerabilities found in open-source applications.

IV. TIMELINE

- **Weeks 1–2:** Infrastructure setup (Tree-sitter, LLM API integration) and implementation of Stage 1 including the Retrieval Layer.
- **Weeks 3–4:** Implementation of Stage 2
- **Weeks 5–6:** Implement stage 3, including the Critic Agent.
- **Weeks 7–8:** Evaluation

REFERENCES

- [1] J. Liu, Y. Yang, K. Chen, and M. Lin, “Generating API Parameter Security Rules with LLM for API Misuse Detection,” *arXiv preprint arXiv:2409.09288*, 2025. Available: <https://arxiv.org/abs/2409.09288>
- [2] Gemini 3 Flash (Nano Banana 2 model), “Fig 2. The Bounded Inter-Procedural Retrieval Layer,” AI-generated image, Google, 2026.
- [3] Gemini 3 Flash (Nano Banana 2 model), “Fig 3. The New Stage 3,” AI-generated image, Google, 2026.